**UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA**

Ingeniería en Sistemas de Información

Sede: Boca del Monte

Curso: Telecomunicaciones I

Ing. Luis Fernando Alvarado Cruz

Sección: A

# Proyecto Final

## Campus SDN: segmentación VLAN y ruteo inter-VLAN definido por software

**Repositorio:** https://github.com/jonathanc4chan/Proyecto-Final-de-Telecomunicaciones

**Nombre**

Jonathan Joel Chan Cuellar 7690-22-1805

**GUATEMALA 29 DE OCTUBRE DEL 2026**

# 1. Introducción

Se construyó una maqueta funcional emulada de la red del nuevo edificio administrativo de Distribuidora Quetzal, S.A. (empresa ficticia). La red usa tres switches Open vSwitch que solo reenvían tramas según reglas de flujo, y un único controlador SDN (os-ken 3.1.1, OpenFlow 1.3) que decide toda la lógica: separación de departamentos en VLAN, ruteo entre ellas y política de seguridad. Todo corre en Mininet dentro de una máquina virtual Ubuntu.

**Enunciado del proyecto:** diseñar, construir y defender la red de un campus corporativo cuyo plano de control vive por completo en un controlador SDN, cumpliendo los requisitos R1 a R6 y, como extensiones opcionales, X1 (red de invitados con límite de 2 Mbps) y X2 (el gateway responde ping).

**Objetivo general:** implementar en un entorno emulado una red de campus segmentada en VLAN, con ruteo inter-VLAN y políticas de acceso, cuyo comportamiento esté definido en su totalidad por una aplicación de control SDN que hable OpenFlow 1.3.

**Objetivos específicos:**

- Reproducir con exactitud la topología dada: nombres, DPID, puertos, direcciones IP y MAC.
- Implementar puertos de acceso y troncales 802.1Q usando únicamente reglas de flujo.
- Implementar un gateway por VLAN y el reenvío IP entre VLAN con el comportamiento observable de un router real.
- Traducir la política de seguridad escrita en reglas de flujo verificables, instaladas desde que los switches se conectan.
- Demostrar con evidencia obtenida en la propia red que cada requisito se cumple.

## 2. Topología y direccionamiento

La red tiene tres switches: **s1** es el núcleo y cumple el papel de switch multicapa; **s2** (piso 1) y **s3** (piso 2) son switches de acceso. Los troncales s1–s2 (puerto 1 con puerto 1) y s1–s3 (puerto 2 con puerto 1) llevan todas las VLAN etiquetadas con 802.1Q, sin VLAN nativa. El controlador escucha en 127.0.0.1:6653.

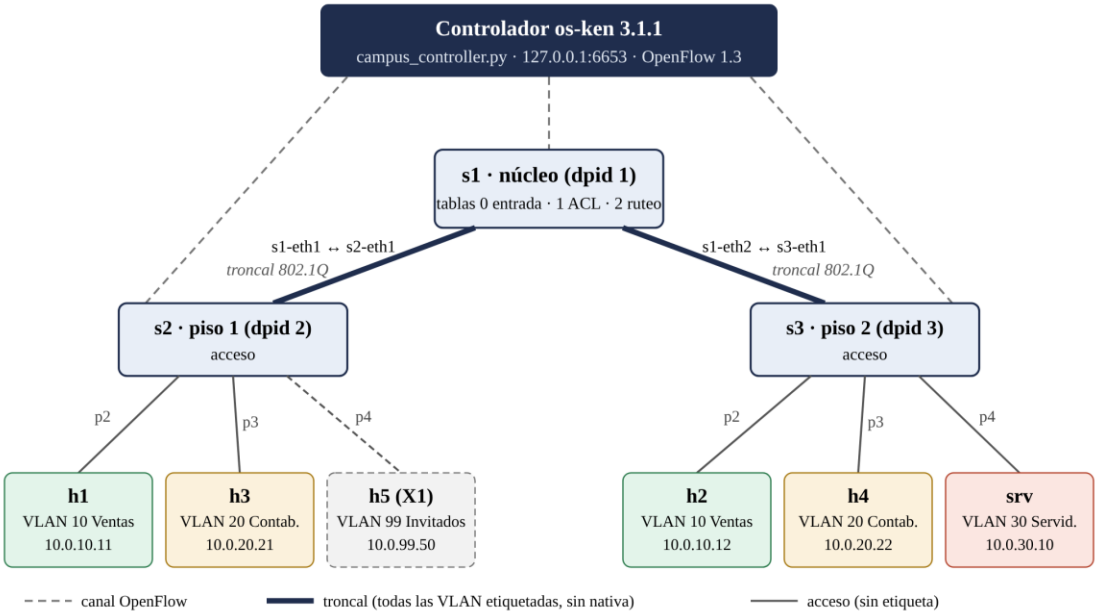



**Figura 1.** Diagrama propio de la red. h5 solo existe si la topología se levanta con --with-h5.

**Tabla 1.** Hosts, direccionamiento y ubicación

| Host | VLAN | IP | MAC | Conexión |
|---|---|---|---|---|
| h1 | 10 Ventas | 10.0.10.11/24 | 00:00:00:00:10:11 | s2, puerto 2 |
| h2 | 10 Ventas | 10.0.10.12/24 | 00:00:00:00:10:12 | s3, puerto 2 |
| h3 | 20 Contabilidad | 10.0.20.21/24 | 00:00:00:00:20:21 | s2, puerto 3 |
| h4 | 20 Contabilidad | 10.0.20.22/24 | 00:00:00:00:20:22 | s3, puerto 3 |
| srv | 30 Servidores | 10.0.30.10/24 | 00:00:00:00:30:10 | s3, puerto 4 |
| h5 (X1) | 99 Invitados | 10.0.99.50/24 | 00:00:00:00:99:50 | s2, puerto 4 |

El gateway de cada VLAN es la dirección .1 de su subred (10.0.10.1, 10.0.20.1, 10.0.30.1 y 10.0.99.1) y todos comparten la MAC 02:00:00:00:00:01. Los hosts no tienen entradas ARP estáticas: resuelven todo con ARP normal.

## 3. Diseño del controlador

Una sola aplicación os-ken (campus_controller.py) instala todas las reglas cuando cada switch se conecta. Antes de instalar, borra las reglas previas del switch (y los meters de s1); por eso la red se recupera sola al reiniciar el controlador. La lógica reactiva se limita al ARP y al ping del gateway: el tráfico de datos nunca sube al controlador.

**s2 y s3 (acceso)**, tabla 0, prioridad 100, dos reglas por VLAN:

- Puerto de acceso: solo tramas sin etiqueta (vlan_vid=OFPVID_NONE). Se hace push_vlan, se fija el VID de la VLAN del puerto y se envía por el troncal; si hay otro puerto de la misma VLAN en el switch, también se entrega allí sin etiqueta.
- Troncal: solo tramas con etiqueta de una VLAN que existe en ese switch. Se hace pop_vlan y se entregan únicamente a los puertos de acceso de esa VLAN.

- Todo lo demás no coincide con ninguna regla y se descarta, porque no hay regla de table-miss. Así una VLAN nunca llega al puerto de otra, ni con broadcast.

**s1 (núcleo)** usa tres tablas: la 0 decide si el paquete va al gateway, la 1 aplica la política y la 2 rutea.

**Tabla 2.** Tablas y prioridades de s1

| Tabla | Prioridad | Qué hace | Tipo |
|---|---|---|---|
| 0 | 300 | ARP request por la IP del gateway de la VLAN: al controlador, que responde con la MAC 02:00:00:00:00:01 | Reactiva |
| 0 | 250 | ICMP echo request al gateway de la propia VLAN (X2): al controlador, que responde el echo reply | Reactiva |
| 0 | 200 | IPv4 dirigido a la MAC del gateway: pasa a la tabla 1 | Proactiva |
| 0 | 150 | Cualquier otra trama dirigida a la MAC del gateway: se descarta | Proactiva |
| 0 | 10 | Misma VLAN: de un troncal al otro conservando la etiqueta, sin ruteo ni cambio de TTL | Proactiva |
| 1 | 100 | Tráfico autorizado por la matriz: pasa a la tabla 2 (el de invitados antes pasa por el meter 1) | Proactiva |
| 1 | 50 | Celdas denegadas de la matriz: se descartan, una regla por celda para tener su contador | Proactiva |
| 1 | 0 | Todo lo demás: se descarta (denegar por defecto) | Proactiva |
| 2 | 100 | Una regla por host destino y puerto de entrada: TTL−1, MAC origen = gateway, MAC destino = host, VLAN del destino y salida | Proactiva |

En total s1 queda con 49 reglas, y s2 y s3 con 6 cada uno. La tabla 2 tiene una regla por puerto de entrada porque un switch OpenFlow no reenvía un paquete por el puerto por el que llegó, salvo con la acción IN_PORT. Cuando origen y destino cuelgan del mismo switch de acceso (por ejemplo h2 y srv en s3), el paquete sube a s1, se rutea y vuelve por el mismo troncal con la etiqueta de la VLAN destino. La ACL va antes del ruteo para descartar sin haber modificado el paquete.

## 4. Cumplimiento de los requisitos R1 a R6

**Tabla 3.** Cumplimiento de los requisitos

| Requisito | Cómo se cumple | Dónde |
|---|---|---|
| R1 Plano de datos | Switches con protocols=OpenFlow13 y failMode=secure: sin controlador no reenvían nada. Controlador remoto en 127.0.0.1:6653. Hosts sin ARP estático. | campus_topo.py |
| R2 Segmentación VLAN | Acceso sin etiqueta y troncal 802.1Q con push_vlan / pop_vlan en s2 y s3, sin VLAN nativa. La misma VLAN cruza de piso en capa 2 por s1 sin tocar el TTL. Una VLAN solo sale a puertos de su VLAN. | s2, s3 y tabla 0 de s1 |
| R3 Ruteo inter-VLAN | El controlador responde el ARP del gateway con 02:00:00:00:00:01. El ruteo ocurre en la tabla 2 de s1: dec_nw_ttl, MAC origen = gateway, MAC destino = host y VLAN del destino; IN_PORT para el mismo switch de acceso. | Tablas 0 y 2 de s1 |
| R4 Política de seguridad | Matriz completa en la tabla 1 de s1, instalada al conectarse el switch. Lo denegado se descarta en el switch, sin consultar al | Tabla 1 de s1 |

| | | |
|---|---|---|
| | controlador. ACL sin estado: Servidores a Ventas permite TCP con puerto origen 80 (respuestas HTTP). | |
| R5 Servicios | Al levantar la topología, srv sirve /tmp/srv_www por HTTP en el puerto 80 y el panel de administración en el 8080. | campus_topo.py |
| R6 Fuente de verdad | Todas las reglas las instala el controlador; al conectarse un switch borra y reinstala todo. La red arranca con dos comandos. | campus_controller.py, README.md |

No se viola ninguna restricción del enunciado: no se usa la acción NORMAL ni las opciones tag / trunks de OVS, no hay ovs-ofctl add-flow ni reglas en el script de topología, los hosts no hacen de router, no hay bridges de Linux y se usa os-ken 3.1.1.

## 5. Extensiones

**X1, red de invitados.** El host h5 (VLAN 99, s2 puerto 4) se agrega con sudo python3 campus_topo.py --with-h5; sin esa opción la red se levanta sin él. La política de invitados vive en la misma tabla 1 de s1:

- Solo se permite TCP al puerto 80 de srv y las respuestas de srv (TCP con puerto origen 80). Se descarta todo lo demás: ICMP, el puerto 8080 y el tráfico hacia o desde las otras VLAN. Nadie puede iniciar comunicación hacia h5.
- Las dos reglas permitidas pasan antes por el meter 1, una banda drop de 2000 kbps con ráfaga de 200 kb. Como cubre ambos sentidos, el límite de 2 Mbps vale para todo el tráfico de invitados.
- Los meters no se borran junto con los flujos; por eso, al conectarse, s1 elimina todos los meters y vuelve a crear el suyo antes de instalar las reglas que lo usan.

**X2, el gateway responde ping.** En la tabla 0 de s1 hay una regla por VLAN que envía al controlador el ICMP echo request dirigido a la IP del gateway de esa VLAN. El controlador arma el echo reply con la IP y MAC del gateway y lo devuelve por el puerto de entrada con un Packet-Out. El ping al gateway de otra VLAN no se responde.

## 6. Evidencias

Las capturas siguientes son reales, tomadas en la máquina virtual con la versión final del código.

### R1: modo seguro y reglas instaladas

Los tres switches responden secure: sin controlador no reenvían. El controlador instaló 49 reglas en s1 y 6 en s2 y s3 (dump-flows agrega una línea de encabezado).

```
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ for s in s1 s2 s3; do sudo ovs-vsctl get-fail-mode $s; done
secure
secure
secure
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1 | wc -l
50
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s2 | wc -l
7
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s3 | wc -l
7
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$
```

**Figura 2.** fail-mode de los tres switches y número de reglas.

### R2: segmentación VLAN

h1 y h2 (VLAN 10, pisos distintos) se comunican en capa 2 con TTL 64, igual que h3 y h4 (VLAN 20): ese tráfico no se rutea.

```
s1 s2 s3
*** Servicios de srv: portal :80, panel :8080
*** Starting CLI:
mininet> h1 ping -c3 h2
PING 10.0.10.12 (10.0.10.12) 56(84) bytes of data.
64 bytes from 10.0.10.12: icmp_seq=1 ttl=64 time=9.42 ms
64 bytes from 10.0.10.12: icmp_seq=2 ttl=64 time=0.201 ms
64 bytes from 10.0.10.12: icmp_seq=3 ttl=64 time=0.324 ms

--- 10.0.10.12 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 0.201/3.314/9.418/4.316 ms
mininet> h3 ping -c3 h4
PING 10.0.20.22 (10.0.20.22) 56(84) bytes of data.
64 bytes from 10.0.20.22: icmp_seq=1 ttl=64 time=36.3 ms
64 bytes from 10.0.20.22: icmp_seq=2 ttl=64 time=0.195 ms
64 bytes from 10.0.20.22: icmp_seq=3 ttl=64 time=1.27 ms
```

**Figura 3.** Ping dentro de la misma VLAN entre pisos, con TTL 64.

En el troncal s1–s2 (captura_troncal_s1_s2.pcap), el filtro vlan.id == 10 muestra solo los ARP y pings entre h1 y h2, y vlan.id == 20 los de h3 y h4. Las tramas miden 4 bytes más (102 en vez de 98 para un ping) por la etiqueta, y ninguna viaja sin etiqueta: no hay VLAN nativa.

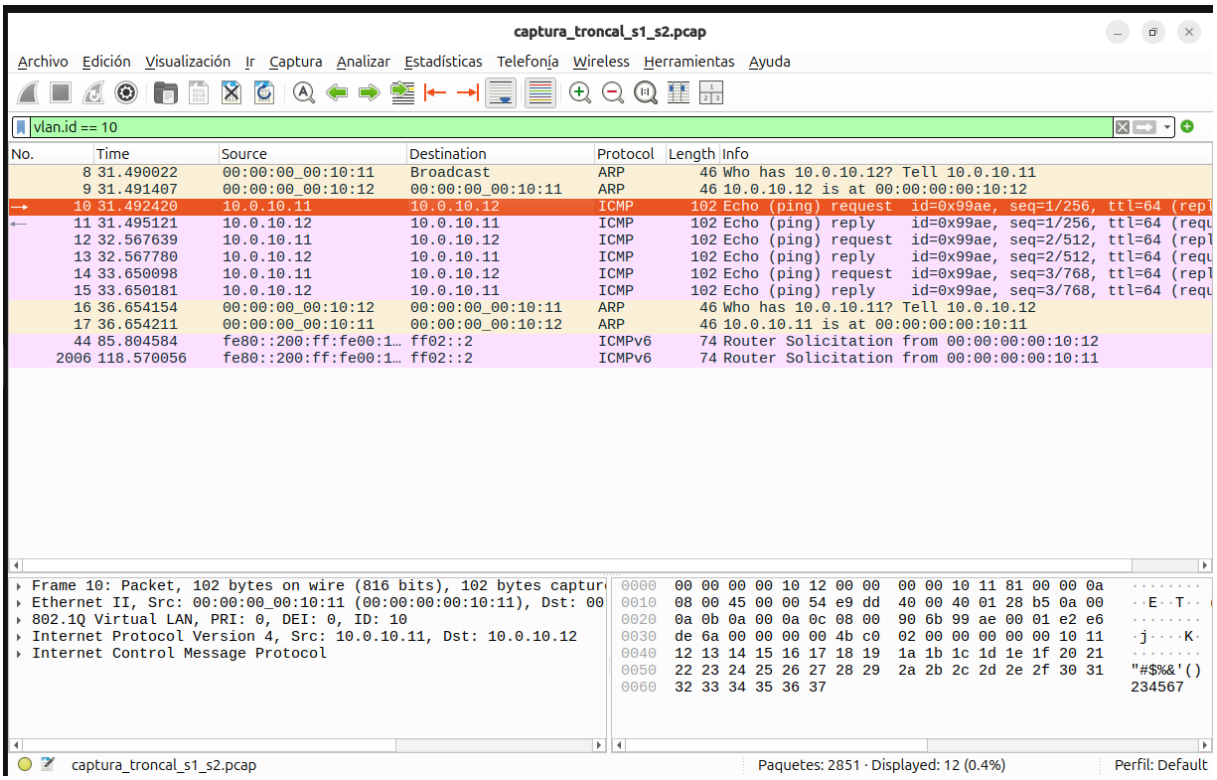



**Figura 4.** Troncal s1–s2 en Wireshark, VLAN 10 (802.1Q ID: 10).

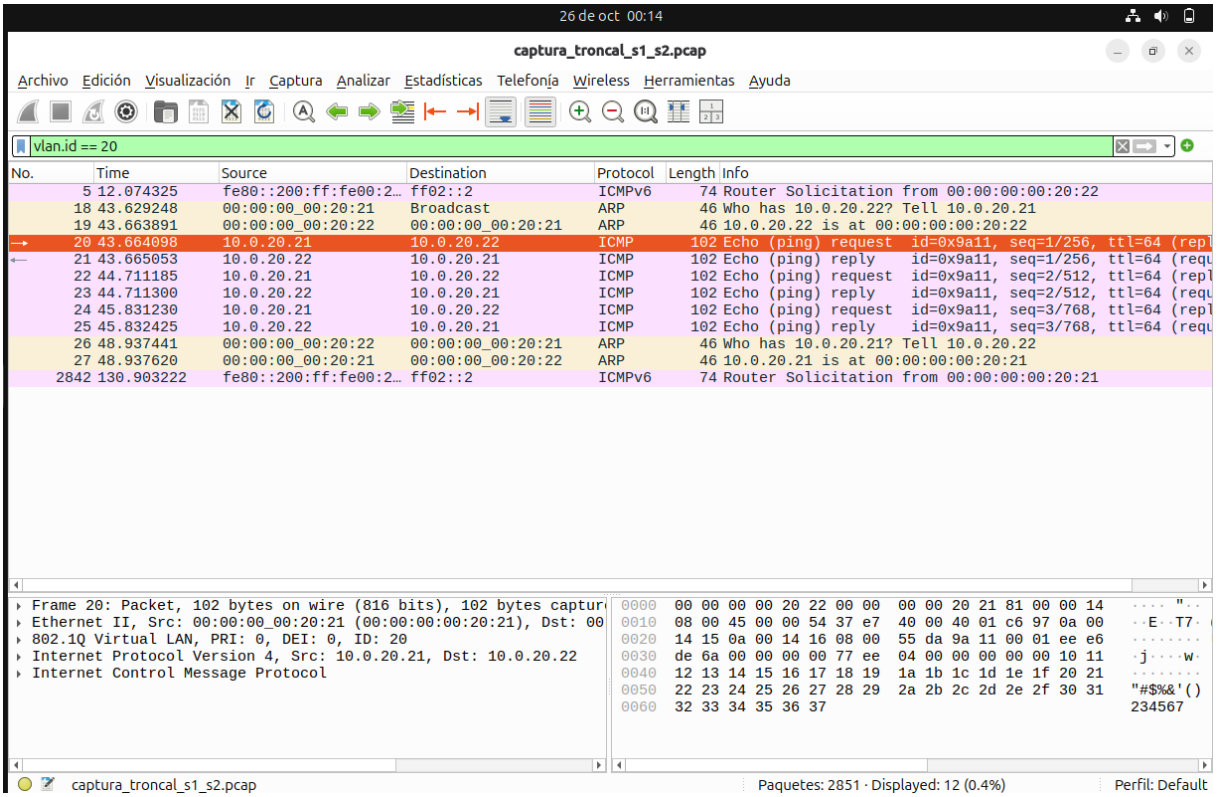



**Figura 5.** Troncal s1–s2 en Wireshark, VLAN 20 (802.1Q ID: 20).

### R3: ruteo inter-VLAN

La tabla ARP de h1 empieza vacía; el ping a srv llega con TTL 63 (un salto de ruteo) y después h1 conoce la MAC del gateway, 02:00:00:00:00:01.

```
(sdn-env) jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo python3 campus_topo.py
*** Configuring hosts
h1 h2 h3 h4 srv
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
*** Waiting for switches to connect
s1 s2 s3
*** Servicios de srv: portal :80, panel :8080
*** Starting CLI:
mininet> h1 arp -n
mininet>  h1 ping -c2 srv
PING 10.0.30.10 (10.0.30.10) 56(84) bytes of data.
64 bytes from 10.0.30.10: icmp_seq=1 ttl=63 time=17.0 ms
64 bytes from 10.0.30.10: icmp_seq=2 ttl=63 time=0.112 ms

--- 10.0.30.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1018ms
rtt min/avg/max/mdev = 0.112/8.549/16.986/8.437 ms
mininet>  h1 ping -c2 srv
PING 10.0.30.10 (10.0.30.10) 56(84) bytes of data.
64 bytes from 10.0.30.10: icmp_seq=1 ttl=63 time=2.94 ms
64 bytes from 10.0.30.10: icmp_seq=2 ttl=63 time=0.072 ms

--- 10.0.30.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1026ms
rtt min/avg/max/mdev = 0.072/1.508/2.944/1.436 ms
mininet> h1 arp -n
Dirección               TipoHW  DirecciónHW         Indic Máscara         Interfaz
10.0.10.1               ether   02:00:00:00:00:01   C                     h1-eth0
mininet>
```

**Figura 6.** ARP del gateway y ping a srv con TTL 63.

La tabla 2 de s1 tiene una regla por host y por troncal de entrada (12 en total). Para srv entrando por s1-eth1: baja el TTL, pone como MAC de origen la del gateway y como destino la de srv, cambia la VLAN a 30 (4126 = 4096 + 30) y sale por s1-eth2. Cuando el destino está detrás del mismo troncal de entrada, la salida es IN_PORT.

```
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1 table=2
 cookie=0x0, duration=49.738s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth1",vlan_tci=0x1000/0x1000,nw_dst=10.0.10.11 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:10:11->eth_dst,set_field:4106->vlan_vid,IN_PORT
 cookie=0x0, duration=49.738s, table=2, n_packets=2, n_bytes=204, priority=100,ip,in_port="s1-eth2",vlan_tci=0x1000/0x1000,nw_dst=10.0.10.11 actions=
dec_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:10:11->eth_dst,set_field:4106->vlan_vid,output:"s1-eth1"
 cookie=0x0, duration=49.738s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth1",vlan_tci=0x1000/0x1000,nw_dst=10.0.10.12 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:10:12->eth_dst,set_field:4106->vlan_vid,output:"s1-eth2"
 cookie=0x0, duration=49.738s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth2",vlan_tci=0x1000/0x1000,nw_dst=10.0.10.12 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:10:12->eth_dst,set_field:4106->vlan_vid,IN_PORT
 cookie=0x0, duration=49.738s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth1",vlan_tci=0x1000/0x1000,nw_dst=10.0.20.21 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:20:21->eth_dst,set_field:4116->vlan_vid,IN_PORT
 cookie=0x0, duration=49.738s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth2",vlan_tci=0x1000/0x1000,nw_dst=10.0.20.21 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:20:21->eth_dst,set_field:4116->vlan_vid,output:"s1-eth1"
 cookie=0x0, duration=49.730s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth1",vlan_tci=0x1000/0x1000,nw_dst=10.0.20.22 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:20:22->eth_dst,set_field:4116->vlan_vid,output:"s1-eth2"
 cookie=0x0, duration=49.730s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth2",vlan_tci=0x1000/0x1000,nw_dst=10.0.20.22 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:20:22->eth_dst,set_field:4116->vlan_vid,IN_PORT
 cookie=0x0, duration=49.730s, table=2, n_packets=2, n_bytes=204, priority=100,ip,in_port="s1-eth1",vlan_tci=0x1000/0x1000,nw_dst=10.0.30.10 actions=
dec_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:30:10->eth_dst,set_field:4126->vlan_vid,output:"s1-eth2"
 cookie=0x0, duration=49.730s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth2",vlan_tci=0x1000/0x1000,nw_dst=10.0.30.10 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:30:10->eth_dst,set_field:4126->vlan_vid,IN_PORT
 cookie=0x0, duration=49.730s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth1",vlan_tci=0x1000/0x1000,nw_dst=10.0.99.50 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:99:50->eth_dst,set_field:4195->vlan_vid,IN_PORT
 cookie=0x0, duration=49.730s, table=2, n_packets=0, n_bytes=0, priority=100,ip,in_port="s1-eth2",vlan_tci=0x1000/0x1000,nw_dst=10.0.99.50 actions=de
c_ttl,set_field:02:00:00:00:00:01->eth_src,set_field:00:00:00:00:99:50->eth_dst,set_field:4195->vlan_vid,output:"s1-eth1"
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$
```

**Figura 7.** Tabla 2 de s1: ruteo con dec_ttl, reescritura de MAC y cambio de VLAN.

### R4 y R5: política de seguridad y servicios

Con h5 levantado, pingall pierde 18 de 30 pares (60 %); sin h5 pierde 8 de 20 (40 %), que son exactamente Ventas con Contabilidad en ambos sentidos. h1 no llega a h3, llega al portal (80) pero no al panel (8080); h3 llega a los dos.

```
srv -> h1 h2 h3 h4 X
h5 -> X X X X X
*** Results: 60% dropped (12/30 received)
mininet> h1 ping -c2 h2
PING 10.0.10.12 (10.0.10.12) 56(84) bytes of data.
64 bytes from 10.0.10.12: icmp_seq=1 ttl=64 time=1.32 ms
64 bytes from 10.0.10.12: icmp_seq=2 ttl=64 time=0.144 ms

--- 10.0.10.12 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1027ms
rtt min/avg/max/mdev = 0.144/0.733/1.323/0.589 ms
mininet> h1 ping -c2 srv
PING 10.0.30.10 (10.0.30.10) 56(84) bytes of data.
64 bytes from 10.0.30.10: icmp_seq=1 ttl=63 time=1.95 ms
64 bytes from 10.0.30.10: icmp_seq=2 ttl=63 time=0.067 ms

--- 10.0.30.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.067/1.008/1.949/0.941 ms
mininet> h1 ping -c2 h3
PING 10.0.20.21 (10.0.20.21) 56(84) bytes of data.

--- 10.0.20.21 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1006ms

mininet> h1 curl -s http://10.0.30.10
<h1>Portal interno - Distribuidora Quetzal</h1>
mininet> h1 curl -s -m3 http://10.0.30.10:8080
mininet> h3 curl -s http://10.0.30.10:8080
<h1>Panel de administracion</h1>
mininet> h1 ping -c2 10.0.10.1
PING 10.0.10.1 (10.0.10.1) 56(84) bytes of data.
64 bytes from 10.0.10.1: icmp_seq=1 ttl=64 time=40.2 ms
64 bytes from 10.0.10.1: icmp_seq=2 ttl=64 time=2.54 ms

--- 10.0.10.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 2.537/21.352/40.168/18.815 ms
mininet>
```

**Figura 8.** pingall, ping con TTL 63, curl a los puertos 80 y 8080 y ping al gateway.

El descarte ocurre en el switch: las 10 reglas de denegación de la tabla 1, más el descarte por defecto, tienen actions=drop y ninguna acción hacia el controlador. Tras un pingall, Ventas↔Contabilidad acumulan 4 paquetes por sentido.

```
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1 table=1 | grep drop
 cookie=0x0, duration=543.396s, table=1, n_packets=4, n_bytes=408, priority=50,ip,nw_src=10.0.10.0/24,nw_dst=10.0.20.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=4, n_bytes=408, priority=50,ip,nw_src=10.0.20.0/24,nw_dst=10.0.10.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.10.0/24,nw_dst=10.0.30.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.30.0/24,nw_dst=10.0.10.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.99.0/24,nw_dst=10.0.10.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.99.0/24,nw_dst=10.0.20.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.99.0/24,nw_dst=10.0.30.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.10.0/24,nw_dst=10.0.99.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.20.0/24,nw_dst=10.0.99.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=50,ip,nw_src=10.0.30.0/24,nw_dst=10.0.99.0/24 actions=drop
 cookie=0x0, duration=543.390s, table=1, n_packets=0, n_bytes=0, priority=0 actions=drop
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$
```

**Figura 9.** Reglas de denegación de la tabla 1 de s1 con sus contadores.

### R6: el controlador es la única fuente de verdad

Con la red levantada se borraron las reglas de los tres switches: cada uno quedó solo con la línea de encabezado (cero reglas). Al reiniciar solo el controlador, los switches se reconectaron y volvieron a 50, 7 y 7 líneas, sin tocar Mininet.

```
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ for s in s1 s2 s3; do sudo ovs-vsctl get-fail-mode $s; done
secure
secure
secure
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1 | wc -l
50
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s2 | wc -l
7
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s3 | wc -l
7
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ for s in s1 s2 s3; do sudo ovs-ofctl -O OpenFlow13 del-flows $s; done
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s3 | wc -l
1
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s3 | wc -l
1
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ for s in s1 s2 s3; do sudo ovs-vsctl get-fail-mode $s; done
secure
secure
secure
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s2 | wc -l
1
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s2 | wc -l
7
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ for s in s1 s2 s3; do sudo ovs-vsctl get-fail-mode $s; done
secure
secure
secure
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s2 | wc -l
7
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s3 | wc -l
7
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1 | wc -l
50
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$
```

**Figura 10.** Borrado de reglas y recuperación tras reiniciar el controlador.

## X1 y X2

Con --with-h5, h5 no hace ping a srv, sí abre el portal por HTTP/80, y la descarga de un archivo de 2 MB tardó 17 s, a unos 118 KB/s (≈ 0,95 Mbps), bajo el límite de 2 Mbps: con un meter que descarta, TCP baja su ritmo y se estabiliza por debajo. En la Figura 8 se ve también que h1 hace ping a su gateway 10.0.10.1 y recibe respuesta (X2).

```
mininet> h5 ping -c2 srv
PING 10.0.30.10 (10.0.30.10) 56(84) bytes of data.

--- 10.0.30.10 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1009ms

mininet> h5 curl -s http://10.0.30.10
<h1>Portal interno - Distribuidora Quetzal</h1>
mininet> srv dd if=/dev/zero of=/tmp/srv_www/grande.bin bs=1M count=2
2+0 registros en
2+0 registros fuera
2097152 bytes (2.1 MB, 2.0 MiB) copiados, 0.00637778 s, 350 MB/s
mininet> h5 curl -o /dev/null -w "%{speed_download}\n" http://10.0.30.10/grande.bin
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  2.00M 100  2.00M    0      0 115.4k      0   00:17   00:17         115.9k
118216
mininet>
```

**Figura 11.** Pruebas de h5: sin ping, portal por HTTP/80 y descarga limitada.

El meter 1 de s1 es de tipo drop con rate=2000 kbps, lo usan las dos reglas de invitados (flow_count:2) y descartó 285 de 3042 paquetes: el límite se aplica.

```
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$ sudo ovs-ofctl -O OpenFlow13 dump-meters s1
   sudo ovs-ofctl -O OpenFlow13 meter-stats s1
[sudo: authenticate] Contraseña:
OFPST_METER_CONFIG reply (OF1.3) (xid=0x2):
meter=1 kbps burst stats bands=
type=drop rate=2000 burst_size=200
OFPST_METER reply (OF1.3) (xid=0x2):
meter:1 flow_count:2 packet_in_count:3042 byte_in_count:2769972 duration:2738.903s bands:
0: packet_count:285 byte_count:446018
jonathan@jonathan-VirtualBox:~/Escritorio/sdn-proyecto./sdn-proyecto$
```

**Figura 12.** dump-meters y meter-stats de s1.

### Canal de control OpenFlow

En captura_control_openflow.pcap (filtro openflow_v4) se ve el arranque de un switch en orden: HELLO en ambos sentidos (negociación de OpenFlow 1.3), FEATURES_REQUEST y FEATURES_REPLY, la descripción de puertos, un FLOW_MOD de borrado seguido de BARRIER_REQUEST/REPLY, y luego los FLOW_MOD de instalación. También aparecen los dos METER_MOD del meter de X1, antes de las reglas que lo usan.

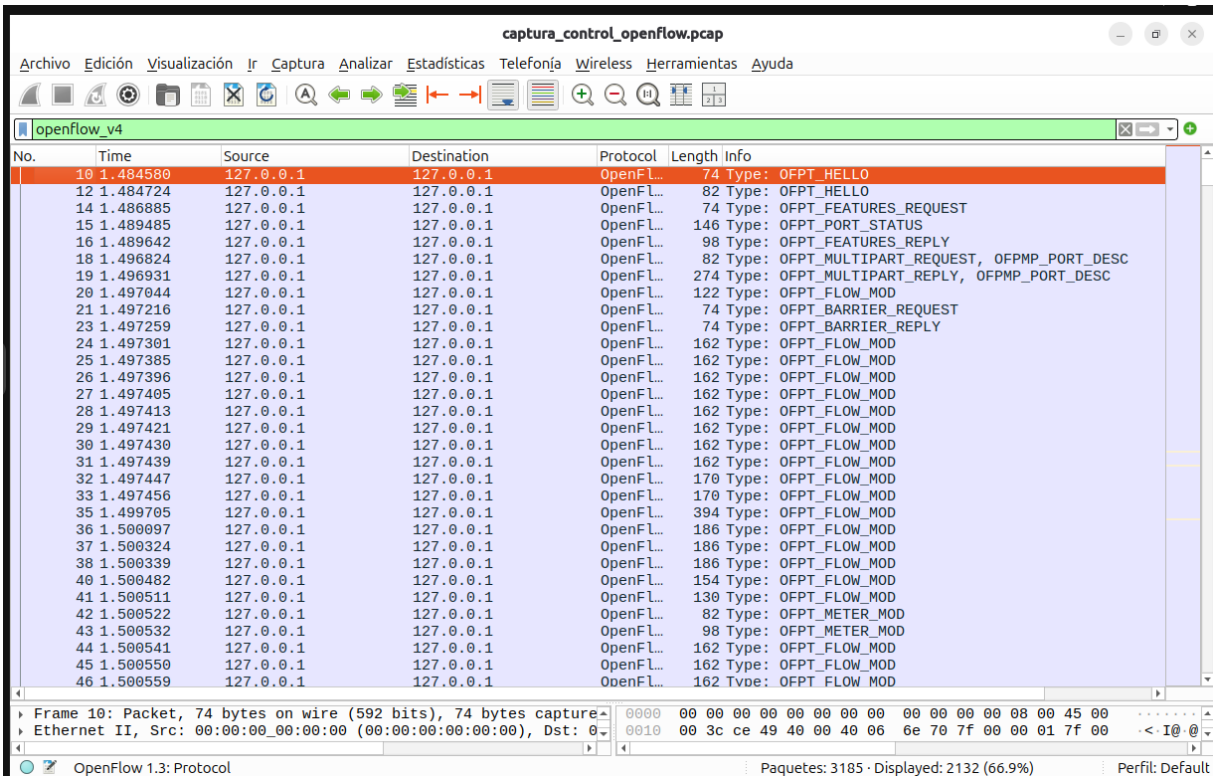



**Figura 13.** Canal de control OpenFlow en Wireshark.

## 7. Problemas encontrados y soluciones

**Tabla 4.** Problemas y soluciones

| Problema | Qué se observó | Solución |
|---|---|---|
| Las rutas de la tabla 2 no se instalaban | El ARP del gateway se respondía, pero el ping entre VLAN perdía el 100 %. s1 tenía 12 reglas en vez de 22: faltaban las 10 de ruteo. | Las reglas cambiaban el VID sin que su match garantizara una trama etiquetada, y OVS lo rechaza. Se agregó vlan_vid con la bandera de etiqueta presente al match de cada ruta. |
| Rechazos de reglas invisibles | os-ken no muestra los errores OpenFlow; el fallo anterior solo se vio al contar reglas. | Manejador de EventOFPErrorMsg que escribe «Error OpenFlow» con el dpid, el tipo y el código. |
| Ruteo en el mismo switch de acceso | Un switch no reenvía un paquete por el puerto por el que entró. | Rutas por puerto de entrada, con la acción IN_PORT cuando el destino cuelga del mismo troncal. |
| ACL sin estado | Permitir solo un sentido corta las respuestas. | Servidores a Ventas permite TCP con puerto origen 80 (respuestas HTTP); Ventas a Servidores, puerto destino 80. |

| | | |
|---|---|---|
| Advertencias al arrancar | Python 3.14 muestra «Eventlet is deprecated» y «RLock(s) were not greened». | Son avisos de la biblioteca eventlet de os-ken 3.1.1; el controlador funciona y no se cambió la versión exigida. |
| Comando de verificación mal usado | ovs-vsctl get-fail-mode acepta un solo switch por llamada. | Se consulta cada switch en un bucle for. |

## 8. Ejecución y entregables

La red arranca con dos comandos, desde la carpeta del proyecto: el controlador osken-manager --ofp-tcp-listen-port 6653 campus_controller.py y, en otra terminal, la topología sudo python3 campus_topo.py (o con --with-h5 para X1). El README.md detalla los pasos y las pruebas.

**Repositorio del código:** https://github.com/jonathanc4chan/Proyecto-Final-de-Telecomunicaciones. Contiene campus_topo.py, campus_controller.py, README.md, captura_troncal_s1_s2.pcap y captura_control_openflow.pcap.

## 9. Conclusiones

1. **Cumplimiento de objetivos:** la red cumple R1 a R6 y las extensiones X1 y X2 con un único controlador que instala todas las reglas al conectarse cada switch, sin NORMAL, sin VLAN de OVS y sin routers en los hosts.
2. **Diseño en tablas:** separar s1 en entrada, política y ruteo permitió cambiar la política sin tocar el ruteo y descartar el tráfico denegado antes de modificarlo.
3. **Plano de datos en los switches:** dejar al controlador solo el ARP y el ping del gateway hace que el tráfico de datos y el descarte ocurran a velocidad de switch, sin consultar al controlador.
4. **Limitación:** la ACL no guarda estado; Servidores a Ventas permite cualquier TCP con puerto origen 80, no solo respuestas. Un cortafuegos con estado lo evitaría, pero OpenFlow 1.3 estándar no incluye seguimiento de conexiones.